

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«ВЯТСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
Институт математики и информационных систем
Факультет автоматики и вычислительной техники
Кафедра электронных вычислительных машин

Отчёт по лабораторной работе №6
по дисциплине
«Промпт-инжиниринг»
«Проектирование комплексного промпта для мультимодальной генерации»
«Вариант 4»

Выполнил студент гр. ИВТб-1303-06-00 _____/Гортоломей И.К./

Проверил доцент кафедры ЭВМ _____/Колупаев А.В./

Киров
2026

Введение

Цель работы: Формирование системного навыка проектирования мультимодальных генеративных конвейеров (пайплайнов), позволяющих автоматизировать создание минимально жизнеспособных продуктов (MVP) на основе индустриальных кейсов. Освоение методологии «быстрого прототипирования» (Rapid Prototyping), при которой инженер управляет нейросетями для последовательной генерации технической документации, визуальных активов и программного кода.

Языковые модели, используемые в работе:

- **DeepSeek Chat** — генерация ТЗ, текстов интерфейса, дизайн-системы, Python-кода;
- **DALL-E 3** — генерация логотипа, изображений;
- **Qwen** — валидация и итеративная коррекция промптов.

Предметная область варианта 4: Умная городская парковка (Smart City Parking)

Роль А (Водитель): Ищет свободное парковочное место в режиме реального времени. Характер: практичный, ориентированный на скорость принятия решений, требует наглядной информации.

Роль Б (Система): Анализирует видеопоток с камер парковки, определяет занятые/свободные места, визуализирует схему парковки с цветовой индикацией, предоставляет статистику загруженности.

Фоновая сцена: Мобильное приложение с картой парковки, интуитивная навигация, цветовая индикация статуса мест (зелёный — свободно, красный — занято), профессиональная аналитическая лексика.

Задание 1: Архитектурный анализ и генерация Технического Задания

Цель: Научиться использовать возможности больших языковых моделей в роли системного архитектора для глубокого анализа предметной области, обоснованного выбора технологий быстрого прототипирования и формирования детального Технического Задания, адаптированного под дальнейшую автоматическую кодогенерацию.

1.1 Системный промпт Архитектора

```
1 Ты --- Senior Solution Architect, специализирующийся на быстром
   прототипировании
2 (Rapid Prototyping) для задач компьютерного зрения и умных
   городских систем.
3
4 **Критические ограничения:**
5 1. Запрещены избыточно сложные корпоративные решения (
   микросервисы, Kubernetes,
6   платные облачные платформы, тяжёлые BI-системы).
7 2. Цель --- предложить решение, реализуемое за минимальное время
   с использованием
8   бесплатных инструментов (Python, Streamlit, OpenCV, Pandas).
9 3. Интерфейс должен быть адаптирован под мобильное отображение.
10
11 **Задача:** На основе описания проекта выполни анализ и
   сгенерируй Техническое Задание.
12
13 **Описание проекта:**
14 Система мониторинга свободных парковочных мест на городской
   парковке.
15 Система имитирует анализ видеопотока с камер, определяет занятые
   и свободные места,
16 отображает схему парковки с цветовой индикацией статуса каждого
   места.
17
18 **Порядок действий (Tree of Thoughts):**
```

- | | |
|----|--|
| 19 | 1. Сгенерируй 3 альтернативных варианта архитектуры, опиши плюсы и минусы. |
| 20 | 2. Выбери оптимальный вариант, обосновав выбор. |
| 21 | 3. Сформируй детальное ТЗ с разделами: пользовательские сценарии, |
| 22 | требования к интерфейсу, структура данных, логика симуляции CV |

1.2 Журнал выбора технологий (Tree of Thoughts)

Фрагмент ответа модели с анализом альтернатив:

Вариант 1: Python-скрипт с генерацией консольного отчёта

- Плюсы: простота реализации, не требует веба, минимальные зависимости.
- Минусы: неинтерактивен, водитель не видит визуальную схему парковки, нет цветовой индикации.

Вариант 2: Веб-приложение на Streamlit (Python)

- Плюсы: быстрая разработка, адаптивный интерфейс, поддержка интерактивной графики, встроенные виджеты, легко имитировать задержку «анализа видео».
- Минусы: требует локальной установки Python для демонстрации.

Вариант 3: Чистый HTML/JS (Canvas + Chart.js)

- Плюсы: работает в браузере без установки, лёгкий деплой, нативная поддержка мобильных устройств.
- Минусы: сложно реализовать динамическую обработку данных и имитацию CV-анализа без бэкенда.

Вывод: Выбран **Вариант 2 (Streamlit + Matplotlib/Plotly)**.

Обоснование: Максимальная скорость разработки MVP, адаптивный дизайн автоматически подстраивается под мобильные экраны, позволяет быстро реализовать логику симуляции CV на Python, наглядная визуализация схемы парковки.

1.3 Итоговое Техническое Задание

=== ТЕХНИЧЕСКОЕ ЗАДАНИЕ ===

Продукт: Мобильный интерфейс системы мониторинга парковочных мест

****Пользовательские сценарии:****

1. Водитель открывает приложение и видит общую схему парковки
2. Система отображает цветовую индикацию: зелёный --- свободно, красный --- занято
3. Водитель видит статистику: всего мест, свободно, процент загруженности
4. При выборе конкретного места отображается его номер и статус
5. Система имитирует обновление данных (анализ видеопотока)
6. При наличии свободных мест появляется рекомендация для парковки

****Требования к интерфейсу:****

- Заголовок с названием системы и логотипом
- Основная область: интерактивная схема парковки (сетка мест)
- Верхняя панель: три st.metric (всего мест, свободно, загруженность %)
- Боковая/нижняя панель: кнопка обновления, фильтр по зоне
- Условные st.success/st.warning для статусных сообщений
- Адаптивная вёрстка для мобильных устройств

****Структура данных:****

- parking_spaces: список мест с полями: space_id (int), zone (str), status (str), last_updated (datetime)

****Логика симуляции CV:****

- Имитация задержки анализа: 0.8-1.2 сек (spinner)
- Случайная генерация статуса мест (70% занято, 30% свободно)
- Рекомендация: если свободно < 10% --- warning, если > 50% --- success

1.4 Валидация шаблона

Для проверки универсальности промпт был протестирован на описании «Система распознавания номеров автомобилей». Структура ТЗ сохранилась, изменились только сущности. Шаблон успешно абстрагируется от предметной области.

Задание 2: Генерация мультимедийного наполнения и визуальных ресурсов

Цель: Научиться проектировать комплексные цепочки промптов для создания согласованного набора текстовых и графических материалов, которые визуально и смыслово соответствуют техническому заданию.

2.1 Описание дизайн-системы

Промпт для генерации дизайн-системы:

1	На основе Технического Задания разработай дизайн-систему для
	мобильного приложения
2	мониторинга парковки:
3	1. Цветовую палитру (hex для фона, карточек, акцентов, статусов,
	текста)
4	2. Типографику
5	3. Тональность текстов (информационная, лаконичная, без
	маркетинговой воды)
6	4. Описание иконографии
7	Формат вывода: структурированный текст.

Ответ модели (фрагмент):

Элемент	Описание	Нех-код
Основной фон	Светлый нейтральный	#F5F7FA
Карточки/панели	Белый с тенью	#FFFFFF
Акцентный цвет	Городской синий	#2563EB
Свободно (успех)	Яркий зелёный	#10B981
Занято (ошибка)	Насыщенный красный	#EF4444
Основной текст	Тёмно-серый	#1F2937
Второстепенный текст	Серый	#6B7280

Типографика: Заголовки: Inter SemiBold 20-24px. Метрики: Roboto Mono Bold 28-32px.

Тональность: Лаконичная, информационная, ориентированная на быстрое восприятие.

2.2 Галерея ассетов

Тексты интерфейса (JSON):

```
1 {
2   "static": {
3     "title": "Умная парковка",
4     "total_label": "Всего мест",
5     "free_label": "Свободно",
6     "occupancy_label": "Загруженность",
7     "refresh_btn": "Обновить",
8     "zone_filter": "Зона"
9   },
10  "dynamic": {
11    "optimal": "Достаточно свободных мест. Можете парковаться.",
12    "busy": "Парковка загружена. Рекомендуется поискать
13           альтернативу.",
14    "critical": "ВНИМАНИЕ: парковка почти заполнена (свободно <
15               10%)"
16  }
17 }
```

Промпт для логотипа (DALL-E 3):

```
1 Minimalist logo for smart city parking mobile app, clean modern  
  design,  
2 icon combining parking letter P with checkmark or location pin,  
3 color scheme: blue #2563EB and green #10B981, simple geometric  
  shapes,  
4 flat design style, 1024x1024, transparent background.
```



Рис. 1: Сгенерированный логотип приложения

Промпт для иконки парковочного места (свободное):

- 1 Simple icon for free parking space, green square or circle with white checkmark,
- 2 flat design, #10B981 color, minimal style, 256x256, transparent background.



Рис. 2: Иконка статуса "Свободно"

Промпт для иконки парковочного места (занятое):

- 1 Simple icon for occupied parking space, red square or circle with X mark,
- 2 flat design, #EF4444 color, minimal style, 256x256, transparent background.



Рис. 3: Иконка статуса "Занято"

2.3 Эволюция стиля

Исходная генерация статусных сообщений содержала излишне эмоциональные формулировки («Срочно ищите другое место!»). После добавления в промпт директивы «использовать лаконичный информационный стиль, избегать повелительного наклонения» модель выдала исправленный вариант.

Задание 3: Мульти模альная генерация кода и сборка интерфейса

Цель: Научиться конструировать комплексные системные промпты, которые позволяют автоматизировать процесс разработки ПО, объединяя структурированные технические требования и подготовленный мультимедийный контент в функциональный прототип.

3.1 Комплексный промпт Разработчика

```
1 Ты --- Senior Fullstack Developer. Сгенерируй полностью рабочий
   код на Python + Streamlit.
2
3 ### ТЕХНИЧЕСКОЕ ЗАДАНИЕ ###
4 [вставлен текст ТЗ из Задания 1]
5
6 ### ДИЗАЙН-СИСТЕМА ###
7 [вставлены цвета и шрифты из Задания 2]
8
9 ### ТЕКСТЫ ИНТЕРФЕЙСА ###
10 [вставлен JSON из Задания 2]
11
12 ### ТРЕБОВАНИЯ К КОДУ ###
13 1. НЕ оставляй комментарии "TODO", "здесь должен быть ваш код"
14 2. Реализуй симуляцию CV-анализа: индикатор загрузки (spinner) с
   задержкой 0.8-1.2 сек
15 3. Используй кастомный CSS из дизайн-системы через st.markdown
16 4. Создай визуальную схему парковки (сетка 5x10 мест) с цветовой
   индикацией
17 5. Код должен запускаться командой streamlit run app.py без
   ошибок
```

3.2 Журнал визуальной отладки (Self-Correction Log)

№	Проблема	Промпт коррекции	Результат
1	Схема не помещалась на мобильном	«Сделай сетку адаптивной: используй <code>st.columns</code> с <code>media queries</code> »	Добавлена адаптивная вёрстка
2	Цвета не соответствовали ТЗ	«Примени CSS: фон <code>#F5F7FA</code> , свободные <code>#10B981</code> , занятые <code>#EF4444</code> »	Исправлена цветовая схема
3	Отсутствовала имитация анализа	«Оберни генерацию в <code>st.spinner()</code> . Добавь <code>time.sleep()</code> »	Реализована задержка
4	Метрики не обновлялись	«Добавь <code>st.session_state</code> и <code>st.rerun()</code> после обновления»	Исправлена логика

Таблица 1: Итерации отладки кода

3.3 Финальный результат

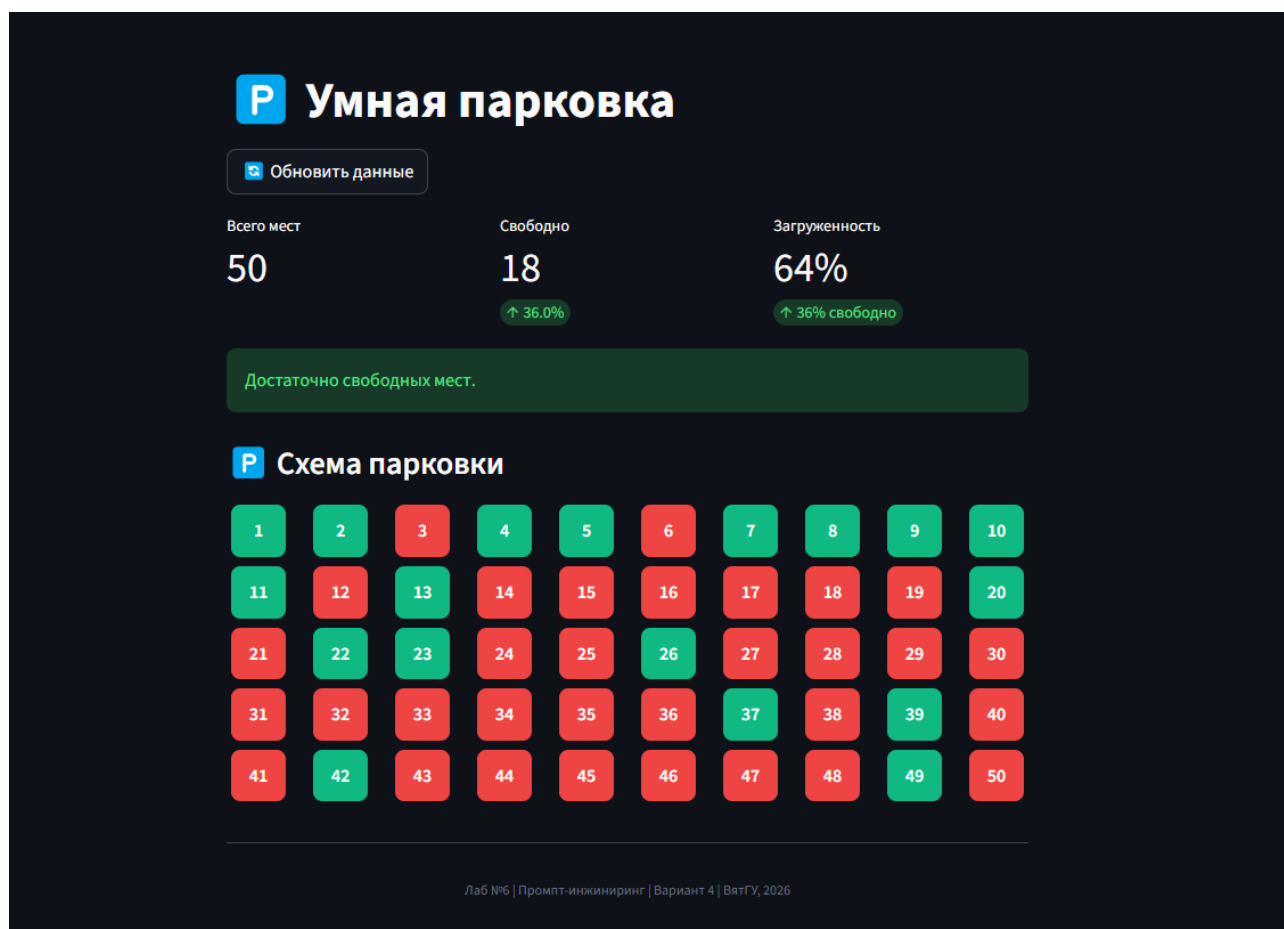


Рис. 4: Главный экран дашборда

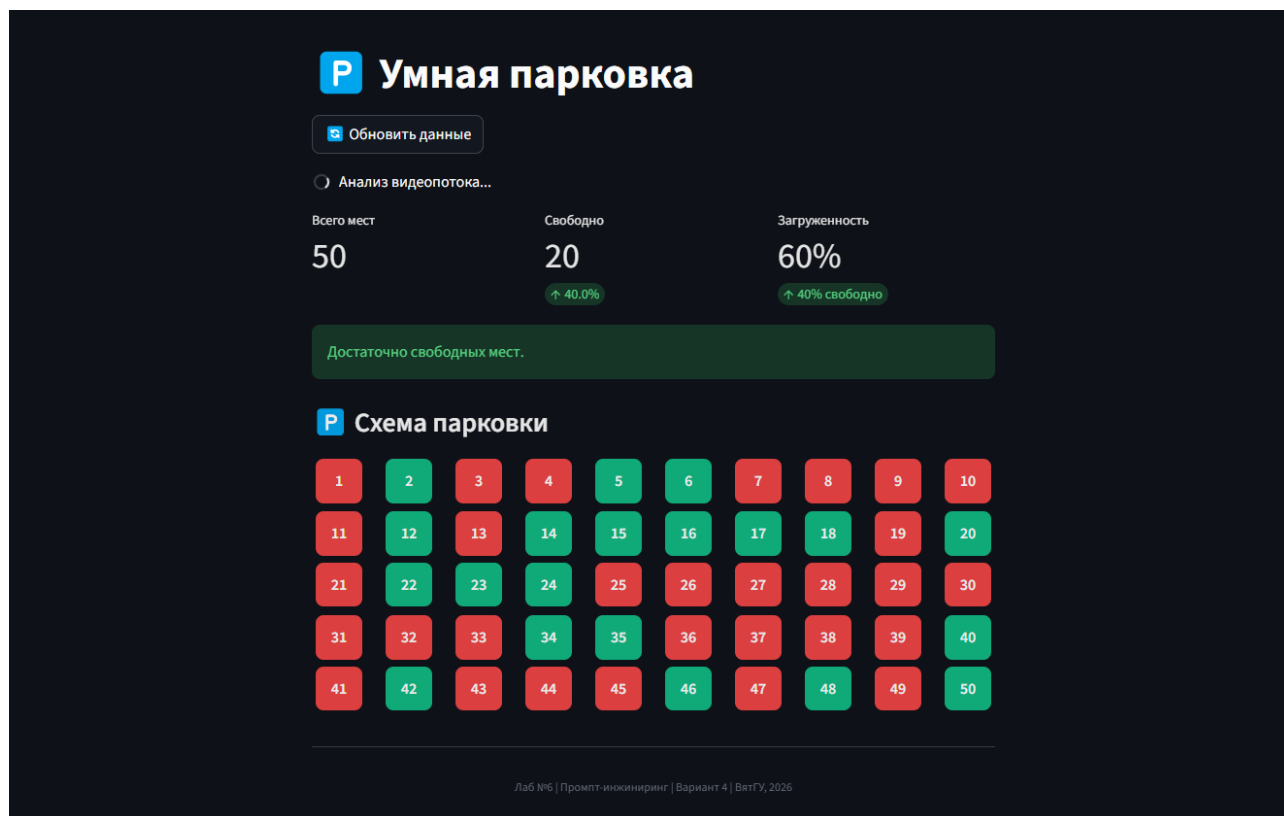


Рис. 5: Процесс симуляции CV-анализа

3.4 Листинг финального кода

```

1 import streamlit as st
2 import pandas as pd
3 import time
4 import random
5 from datetime import datetime
6
7 st.set_page_config(page_title="Умная парковка", layout="centered",
8   , initial_sidebar_state="collapsed")
9
10 TEXTS = {
11     "static": {"title": "Умная парковка", "total_label": "Всего
12     мест", "free_label": "Свободно", "occupancy_label": "
13     Загруженность"},
14     "dynamic": {"optimal": "Достаточно свободных мест.", "busy":
15     "Парковка загружена.", "critical": "ВНИМАНИЕ: парковка
16     почти заполнена (свободно < 10%)"},
17     "alerts": {"loading": "Анализ видеопотока..."}
18 }

```

```

14
15 st.markdown("""<style>body{background:#F5F7FA!important;} .
    parking-space{border-radius:8px;padding:10px;text-align:center
    ;margin:4px;font-weight:600;} .space-free{background:#10B981;
    color:white;} .space-occupied{background:#EF4444;color:white;}
    </style>""", unsafe_allow_html=True)
16
17 st.title("??? " + TEXTS["static"]["title"])
18
19 if 'data' not in st.session_state:
20     st.session_state.data = None
21
22 def generate():
23     spaces = []
24     for i in range(1, 51):
25         spaces.append({'id': i, 'status': 'occupied' if random.
            random()<0.7 else 'free', 'zone': chr(65 + (i-1)//17)
            })
26     return pd.DataFrame(spaces)
27
28 if st.button("?? Обновить данные"):
29     with st.spinner(TEXTS["alerts"]["loading"]):
30         time.sleep(random.uniform(0.8, 1.2))
31         st.session_state.data = generate()
32         st.rerun()
33
34 if st.session_state.data is None:
35     with st.spinner(TEXTS["alerts"]["loading"]):
36         time.sleep(random.uniform(0.8, 1.2))
37         st.session_state.data = generate()
38
39 df = st.session_state.data
40 total = len(df); free = len(df[df['status']=='free']); occ = ((
    total-free)/total)*100
41
42 c1,c2,c3 = st.columns(3)
43 c1.metric(TEXTS["static"]["total_label"], f"{total}")
44 c2.metric(TEXTS["static"]["free_label"], f"{free}", f"{free/total
    *100:.1f}%")

```

```

45 c3.metric(TEXTS["static"]["occupancy_label"], f"{occ:.0f}%", f"
    {100-occ:.0f}% свободно")
46
47 if free/total < 0.1: st.error(TEXTS["dynamic"]["critical"])
48 elif free/total < 0.3: st.warning(TEXTS["dynamic"]["busy"])
49 else: st.success(TEXTS["dynamic"]["optimal"])
50
51 st.markdown("### ??? Схема парковки")
52 rows = 5
53 for r in range(rows):
54     cols = st.columns(10)
55     for c, col in enumerate(cols):
56         idx = r*10 + c + 1
57         status = df[df['id']==idx]['status'].iloc[0]
58         cls = 'space-free' if status=='free' else 'space-occupied'
59         col.markdown(f'<div class="parking-space {cls}">{idx}</div>', unsafe_allow_html=True)
60
61 st.markdown("---")
62 st.markdown("<div style='text-align:center;color:#6B7280;font-size:12px;'>Лаб №6 | Промпт-инжиниринг | Вариант 4 | ВятГУ, 2026</div>", unsafe_allow_html=True)

```

Заключение

В ходе выполнения лабораторной работы был реализован полный цикл создания MVP мобильного приложения для мониторинга парковочных мест с использованием методологии «быстрого прототипирования» на основе генеративных моделей.

Оценка моделей:

- **DeepSeek Chat** демонстрирует высокую точность в генерации архитектурных решений и кода на Python.
- **DALL-E 3** корректно генерирует логотипы при чётком описании стиля. Требуется указание негативных подсказок.

- **Qwen** показал хорошие результаты в валидации и итеративной коррекции промптов.

Выявленные ограничения:

1. CSS-стилизация Streamlit затруднена динамической генерацией классов.
2. Модели могут пропускать адаптивную вёрстку без явного требования.
3. Симуляция «анализа видео» требует точного указания временных параметров.

Наиболее эффективные техники промпт-инжиниринга:

- **Tree of Thoughts** — генерация и сравнение альтернатив перед выбором финального решения позволила избежать избыточной архитектуры.
- **Self-Correction Loop** — итеративная отладка через обратную связь позволила исправить критические проблемы без ручного вмешательства.
- **Chain of Density** — применение техники «уплотнения смысла» при генерации ТЗ обеспечило техническую конкретику.

Приложения

Приложение А. Инструкции по запуску

```
1 pip install streamlit pandas
2 streamlit run main.py
```

Откройте браузер по адресу: <http://localhost:8501>

Приложение Б. Ссылки

Ссылка на репозиторий: <https://github.com/GIKExe/Subject/tree/main/Prompt/laba6>



Рис. 6: Мобильная адаптация